

Efficient Point Cloud Video Recognition via Spatio-Temporal Pruning for MEC-Based Consumer Applications

Cen Chen^{ID}, Senior Member, IEEE, Songyu Xie^{ID}, Graduate Student Member, IEEE, Jing Liu^{ID},
Zhongyao Cheng, Ziqian Zeng, and Huiping Zhuang^{ID}, Member, IEEE

Abstract—3D point cloud videos are widely used in edge computing scenarios to achieve seamless connectivity and extremely low latency in applications such as autonomous driving. It faces difficulties primarily because point cloud data is often voluminous and contains redundancies. These issues can lead to significant drawbacks in practical applications, causing inference delays and reducing accuracy. To tackle these challenges, we propose a novel approach: a transformer-based spatio-temporal pruning method designed specifically for point cloud videos. Our goal is to expedite network inference. Initially, we utilize the attention mechanism of the transformer to calculate importance scores for different frames and tokens within 3D point cloud videos. To capture the temporal context of each point cloud token, we introduce a frame-wise relation matrix for importance score computation. Subsequently, we use these importance scores as criteria for pruning at both frame and token levels. The experimental results indicate that our approach outperforms other methods. For example, on NTU RGB+D 60 dataset, it shows improvement in accuracy by 0.5% and the average inference time decrease by 28%, while memory consumption is reduced by 28%.

Index Terms—Deep learning, point cloud video, dual-scale pruning, mobile edge computing, transformer.

I. INTRODUCTION

PPOINT clouds are typically generated through laser scanning and photogrammetry, possessing irregularity and disorder, which make them difficult to process directly in

traditional neural networks. Neural networks designed for point cloud processing are generally referred to as point cloud neural networks (PCNN) [1], [2]. With the continuous evolution of consumer electronics, 3D point cloud videos have been extensively applied in various domains of mobile edge computing (MEC) based consumer applications, including 3D modeling, virtual reality, augmented reality, autonomous driving, and robotics. MEC based on consumer applications demands network efficiency and fast processing of resources to handle 3D point cloud videos and accomplish tasks more swiftly [3]. The efficient processing of the network can further reduce latency and is extensively studied in MEC for applications such as traffic prediction [4], [5]. For example, in the field of autonomous driving, it is necessary for autonomous vehicles to analyze and process scanned scene data more quickly, while avoiding obstacles and planning routes, in order to prevent serious irreversible losses [6]. However, due to limited computing resources, deploying point cloud neural networks on mobile devices in edge computing for efficient processing remains a challenging task.

In the realm of accelerating point cloud neural networks, some studies resort to voxelization of 3D point cloud videos followed by 2D processing to hasten network computations. For instance, [8] expedited network calculations by channel pruning after voxelizing point clouds. Although this approach achieved pruning, it caused feature loss during point cloud data preprocessing and didn't directly focus on point-wise features. Presently, point-based processing in point cloud neural networks is gaining traction, allowing direct network processing of irregular 3D point cloud data. Regarding pruning in point-based point cloud neural networks, it primarily involves implementations based on the PCL (Point Cloud Library) [9] for pruning, as well as multidimensional pruning [10] and 3D pruning [11] based on convolutional neural networks. However, these methods relying on conventional convolutional neural networks struggle to capture global information, exhibit weak long-range dependencies, and encounter difficulties in achieving parallel computations. Given the current advancement in the Internet of Things (IoT), meeting real-time requirements might pose a challenge and might not cater to present scenario demands and consumer needs.

Recently, transformer-based attention mechanisms allow for the focus on global information and feature extraction from point cloud data [12], [13]. Building on this, we propose a

Manuscript received 1 August 2023; revised 11 January 2024; accepted 5 February 2024. Date of publication 12 February 2024; date of current version 26 April 2024. This work was supported in part by the Fundamental Research Funds for the Central Universities under Grant x2wjD2230230, and in part by the Natural Science Foundation of Guangdong Province of China, and sponsored by CCF-Phytium Fund and CAAI-MindSpore Open Fund, developed on OpenI Community. (Corresponding author: Jing Liu.)

Cen Chen is with the School of Future Technology, South China University of Technology, Guangzhou 510641, China, and also with Pazhou Laboratory, Guangzhou 510330, China (e-mail: chencen@scut.edu.cn).

Songyu Xie and Jing Liu are with the College of Computer Science and Technology, Wuhan University of Science and Technology, Wuhan 430065, China, and also with the Hubei Province Key Laboratory of Intelligent Information Processing and Real-Time Industrial System, Wuhan 430081, China (e-mail: john@wust.edu.cn; luijing_cs@wust.edu.cn).

Zhongyao Cheng is with the Institute for Infocomm Research, Agency for Science, Technology and Research, Singapore (e-mail: cheng_zhongyao@i2r.a-star.edu.sg).

Ziqian Zeng and Huiping Zhuang are with the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology, Guangzhou 510641, China (e-mail: hpzhuang@scut.edu.cn; zqzeng@scut.edu.cn).

Digital Object Identifier 10.1109/TCE.2024.3364677

spatio-temporal dual-scale point cloud video pruning method based on transformer. This method aims to facilitate point cloud data pruning within the transformer, enabling a comprehensive understanding of the data. First, we utilize the attention mechanism of the transformer to calculate the importance scores for each token and frame in the 3D point cloud video, and use these scores as the criteria for the pruning decision without introducing extra parameters. To model the temporal information further, we present a frame-wise relation weight matrix to preserve the temporal relationships among tokens. Secondly, we perform fine-grained time and space dual-scale pruning. On the time scale, we execute a top-K operation based on the importance scores of frames to prune out unimportant frames, which accelerates the computation. On the space scale, we perform a top-K pruning of tokens based on their importance scores, achieving further refinement in pruning. The dual-scale pruning in time and space enables us to remove redundant frames and tokens in the transformer. The experimental results demonstrate that the proposed method can effectively improve accuracy and accelerate network inference.

The main contributions of this paper are as follows:

- The method proposed in this paper is a specialized transformer-based approach for 3D point cloud video pruning. To the best of our knowledge, there has been no prior work on point cloud video pruning specifically using transformer-based methods.
- We propose a spatio-temporal pruning method. Based on the attention mechanism of transformer, we first perform top-K pruning on the frames of the input 3D point cloud video sequence at the temporal scale, and then perform top-K pruning on the tokens of different frames at the spatial scale. Since our pruning is based on the attention mechanism of transformer, we directly use the attention mechanism of transformer as the judgment criterion for pruning, without introducing additional parameters and network structures.
- We present a frame-wise relation weight matrix to model the temporal information of tokens. By introducing the frame-wise relationship matrix, our method can achieve a more accurate modeling of the relationships between frames.
- The experimental results indicate that our approach outperforms other methods on MSR Action 3D dataset and NTU RGB+D 60 dataset showing improvements in accuracy by 0.34% and 0.5% respectively. Additionally, the average inference time are decreased by 30% and 28%, while memory consumption reduced by 22% and 28% respectively on these datasets.

We organize the remainder of this paper as follows. In Section II, we introduce some related researches on pruning and point cloud pruning in the field. In Section III, we mainly describe the network structure and the pruning method used in this paper. Section IV is the experimental part of this paper, which introduces the corresponding datasets and parameter settings, and displays the various indicators and effects of the pruned network, followed by relevant analysis. Finally, Section V is the conclusion part of this paper.

II. RELATED WORK

In this section, we introduce relevant researches on different methods of pruning, and they are mainly partitioned into two categories as follows.

A. Vision Transformer Pruning

Recently, vision transformers [14] have gained widespread application in computer vision tasks. However, many proposed transformers impose substantial demands on computing resources, restricting their deployment in real-world scenarios. Consequently, there is a need for vision transformer pruning. Zhu et al. [15] present a vision transformer pruning approach, which identifies the impacts of dimensions in each layer of transformer and then executes pruning accordingly. Yu et al. [16] propose a Width & Depth Pruning framework that reduces both width and depth dimension. Yu and Wu [17] propose a novel method called UP-ViTs to prune ViTs in a unified manner. Their framework can prune all ViT components and their variants, maintain the models' structure, and generalize well into downstream tasks. Zhang et al. [18] reduce embedding dimensionality by introducing control coefficients. However, these token sampling methods are unstructured and are difficult to use in downstream tasks (e.g., object detection). To address this issue, Zheng et al. [19] introduce joint importance, which integrates essential structural-aware interactions between components for the first time, to perform collaborative pruning. Instead of training full ViTs, Chen et al. [20] integrate sparse training into the transformer to train the smaller subnetwork from scratch. Some works [21], [22], [23], [24] extract data-related redundancy and remove unnecessary image patches. But these pruning-patch methods may compromise the spatial structure of Vision Transformers (ViTs). To preserve this spatial structure, the works [25], [26], [27], [28], [29] studied how artificial intelligence efficiently executes various downstream tasks in the field of computer vision. Yang et al. [30] first show that pruning all components reaches better acceleration. Kong et al. [31] design a dynamic attention-based multi-head token selector, which is a lightweight module for adaptive instance-wise token selection. Song et al. [32] propose a cascade pruning framework named CP-ViT by predicting sparsity in ViT models progressively and dynamically to reduce computational redundancy while minimizing the accuracy loss. Fayyaz et al. [33] introduce a distinguishable, parameter-less adaptive token sampling method to prune image tokens while adhering to specified GFLOPS limitations. A-Vit [34] introduce stopping scores to determine token pruning by accumulating stop scores. Ma et al. [35] introduced a marked pruning pose transformer for 2D human pose estimation. This approach achieves accelerated inference in human pose estimation, enabling the location of a rough human mask and performing self-attention only within selected tokens. The aforementioned methods focus on pruning images, but given the advancements in technology, they may not fully meet the requirements for processing point cloud videos in 3D scenes.

B. Point Cloud Video Pruning

Different from traditional images and videos, point cloud videos offer a more precise representation of three-dimensional scenes and objects, capable of capturing and presenting scene information comprehensively. However, due to issues with capturing equipment and acquisition protocols, most obtained point cloud videos contain redundancies and noise. Consequently, the development of cropping techniques specifically tailored for point cloud videos has gradually emerged to address these challenges. The MPEG point cloud compression platform performs geometry-based and video-based point cloud compression on different forms of point cloud data [36], [37], and accelerates computation by voxelizing 3D point clouds into 2D data. Huang et al. [38] propose CP³, which is a Channel Pruning Plug-in for Point-based network. This work still compresses 3D into 2D for processing. Lee et al. [37] propose a new weight pruning technique for 3D point cloud based on spatial point distribution which difficult to accelerate in practice. Li et al. [39] propose a novel artificial intelligence powered end-to-end framework for efficient point cloud compression. Guo et al. [10] achieve efficient processing by designing a framework specifically for accelerating point cloud neural networks, and Guo et al. [11] use different computational complexities to process different point cloud frames and points for efficient recognition. The above methods of point cloud pruning are all based on convolutional neural networks. When compared to Vision Transformers (ViTs), CNNs exhibit lower efficiency. Efficiency and accuracy are crucial in practical applications to realize real-time processing. To address this issue, our proposed transformer-based spatio-temporal pruning achieve fine-grained pruning of 3D point cloud videos in a parallel computing manner, increasing the efficiency of detection.

III. METHOD

This section primarily introduces the network architecture based on transformer and the proposed spatio-temporal pruning method for processing point cloud videos. Table I lists key notations and definitions used in this paper.

A. The Structure of the Transformer-Based Neural Network

Figure 1 shows the schematic representation of the network architecture employed in this paper. It mainly comprises the point cloud convolutional layer, the transformer layer, and the MLP (Multi-Layer Perceptron) layer.

1) *Point Cloud Convolutional Layer*: Its main function is to extract features from the point cloud data. We refer to the convolution method of traditional 2D convolutional layers and use the displacement feature of points for convolution [13].

For point cloud videos, we first convert them into point frames, and then process each point frame individually. Since the point cloud video is large, there exists redundancy between frames, and each frame can contain thousands or even tens of thousands of points depending on the scene. Directly sampling will not only consume a lot of computing resources and time, but also result in the loss of original video information.

TABLE I
NOTATIONS AND DEFINITIONS

Notations	Definitions
L	The number of fused frames
N	The number of point cloud tokens in each fused frame
$F_l^{(x,y,z)}$	Features of the fused point cloud tokens after feature fusion
E	The local temporal and spatial region around the point (x, y, z, t)
f_l	Temporal radius
C_s	Spatial radius
Q, K, V	The query, key, and value for the layer of transformer
S	The attention score matrix of point cloud
R	The weighted attention score matrix of point cloud
W_Q, W_K, W_V	The weights of the query, key, and value matrices for the layer of transformer
$\mathcal{T}$	The input to the transformer layer
W_{st}^{ij}	The normalized weight of the point cloud token i in the frame s querying the point cloud token j in the frame t
w_{st}^{ij}	The weight of the point cloud token i in the frame s querying the point cloud token j in the frame t
A	The attention score matrix introduced the frame-wise relationship matrix
W'_s	The normalized frame-wise relationship matrix
H	The number of attention heads
S_t^j	The importance score of point cloud token j in frame t
$g_t^j(h)$	The importance score of point cloud token j in frame t within attention head h
$A_{st}^{ij}(h)$	The attention score value of the point cloud token i in frame s querying the point cloud token j in frame t within head h
S_t	The importance score of the fused frame t
O_t	The output after temporal pruning
O_s	The output after spatial pruning

To reduce the inference time of the network and preserves the original video information, one solution is to fuse the information of consecutive point frames and the information of points belonging to frames. We call a frame that fuses the information of the two frames before and after the frame fusion frame. The fusion point is the point of the fusion frame, which integrates the information of points on corresponding positions of the two before and after frames. For convenience, we also refer to a fusion point as a point cloud token.

In the point cloud convolutional layer, first, we choose even numbered frames as key frames. Second, employing the farthest point sampling method to select a specified number of points as key points for each frame. Each key point serves as a center for selecting surrounding key points that have been already sampled and the key points from the corresponding regions of the previous and subsequent frames as the feature fusion area. Third, extracting features from the feature fusion areas, obtaining fusion frames and fusion points (i.e., tokens).

Figure 2 illustrates the concise process of obtaining point cloud tokens through point cloud convolution. Initially, we select key frames from the point cloud video along with their corresponding key points. Subsequently, employing point cloud convolution, we obtain fused frames which incorporate

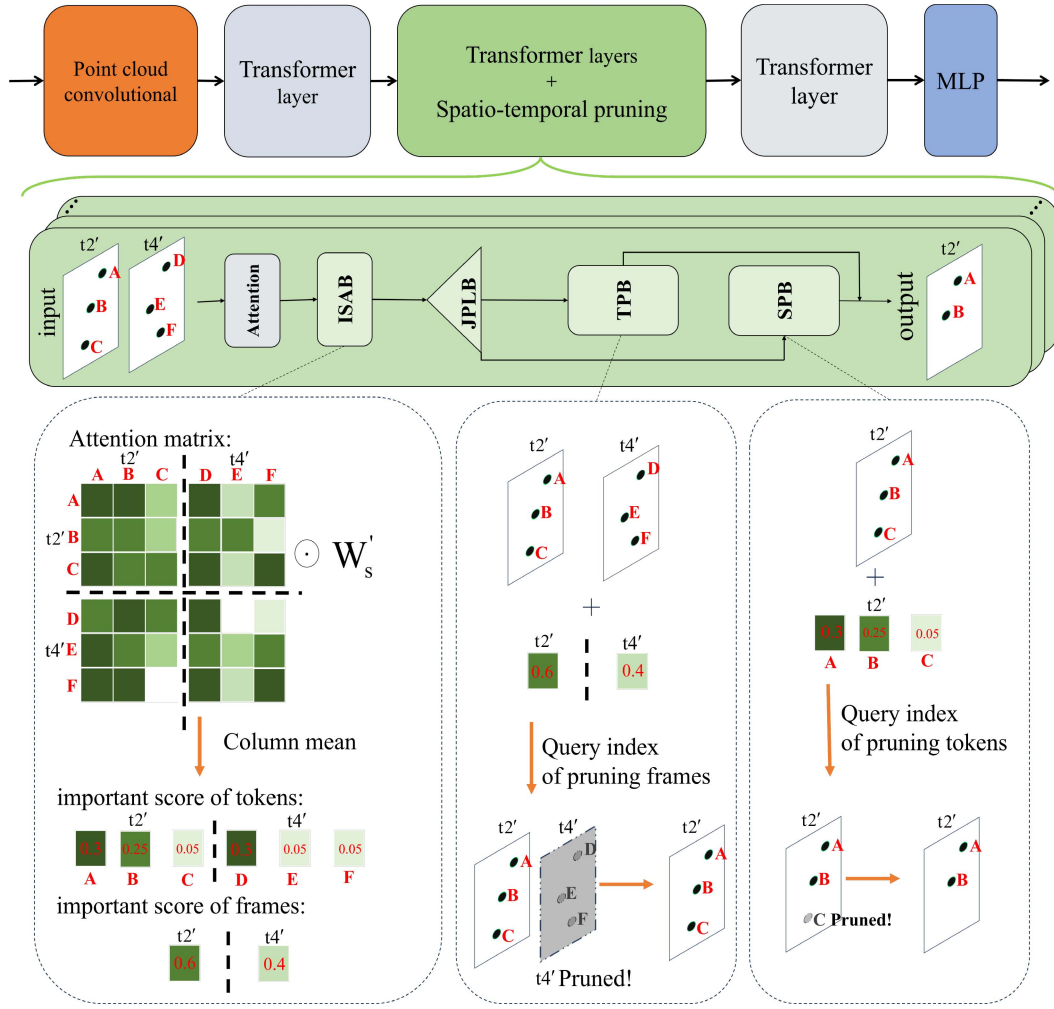


Fig. 1. The schematic diagram of the network architecture and the flowchart of our proposed methodology. The proposed network structure encompasses point cloud convolutional layer, transformer layers, transformer layers with spatio-temporal pruning and MLP (Multi-Layer Perceptron) layer. The integration of self-attention mechanism with the proposed spatio-temporal dual-scale pruning method enables efficient pruning. The transformer layers with spatio-temporal pruning consists five key blocks. The attention block (**Attention**) processes the input fusion frames and point cloud tokens to obtain the corresponding attention score matrices. The importance score allocation block (**ISAB**) calculates the importance scores for each fusion frame and each point cloud token. To further model the inter-frame relationships, this paper introduces the frame-wise relation matrix for importance score computation. The judge pruning layer block (**JPLB**) is responsible for determining whether the current layer should undergo temporal pruning, spatial pruning. The temporal pruning block (**TPB**) and spatial pruning block (**SPB**) execute the main process of the spatio-temporal dual-scale pruning method proposed in this paper. By processing the input point cloud video, different pruning processes are performed at various transformer layers, achieving network acceleration.

contextual information from both preceding and subsequent key frames, as well as fused points (or fusion points) which encompass crucial contextual information from the surrounding points of the key points.

Denote $\mathcal{V} = \{(\mathcal{L}_1, \mathcal{F}_1), (\mathcal{L}_2, \mathcal{F}_2), \dots, (\mathcal{L}_l, \mathcal{F}_l), \dots, (\mathcal{L}_L, \mathcal{F}_L)\}$ to be the input information of a video, where $\mathcal{L}_l \in \mathbb{R}^{3 \times N}$ and $\mathcal{F}_l \in \mathbb{R}^{C \times N}$ represent the coordinate and feature information of the sampled points in the l -th frame, $1 \leq l \leq L$, respectively. N denotes the number of points in the l -th frame, 3 represents the coordinate information in the 3D Cartesian system (i.e., x , y , and z), and C represents the number of other feature information (i.e., normals and colors etc.) of points. The initial feature of the points includes their position information (i.e., x , y , and z) and the corresponding time information in different frames (i.e., l).

Let $F_l^{(x,y,z)} \in \mathbb{R}^{C' \times N'}$ represent the feature information of the sampled point at location (x, y, z) in the frame l by jointly

the feature information of points in the range of C_s . We have

$$F_l^{(x,y,z)} = \sum_{(\Delta x, \Delta y, \Delta z, \Delta l) \in E} W_d \cdot F_{l+\Delta l}^{(x+\Delta x, y+\Delta y, z+\Delta z)} \\ = \sum_{\Delta l = -f_l}^{f_l} \sum_{\|(\Delta x, \Delta y, \Delta z)\| \leq C_s} W_d \cdot F_{l+\Delta l}^{(x+\Delta x, y+\Delta y, z+\Delta z)}, \quad (1)$$

$$W_d = W \cdot (\Delta x, \Delta y, \Delta z, \Delta l)^T, \quad (2)$$

$$\begin{cases} \Delta x = x - x' \\ \Delta y = y - y' \\ \Delta z = z - z' \\ \Delta l = l - l' \end{cases} \quad (3)$$

where E is the spatial and temporal local area of the current sampled token, determined by the spatial radius C_s and temporal radius f_l . $W_d \in \mathbb{R}^{C' \times C}$ denotes the feature weight, and is determined by the displacement of the corresponding

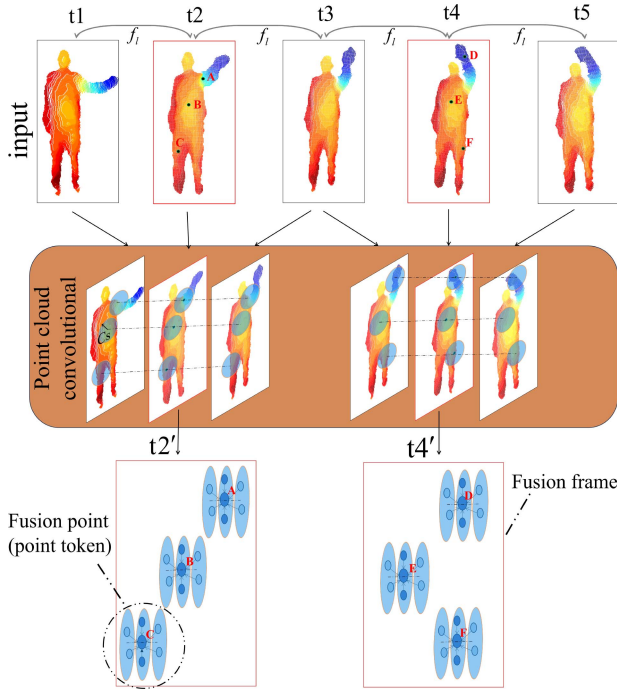


Fig. 2. The process of the point cloud convolutional layers. During the convolution process, key frames from the point cloud video, along with their corresponding key points, are first sampled. Subsequently, by fusing the information from the preceding and succeeding frames of the key frames and incorporating the information within a specified radius around the key points, fused frames and point cloud tokens are generated, encompassing contextual information from the surrounding points.

point. $(\Delta x, \Delta y, \Delta z, \Delta l)$ represents spatial-temporal displacement. $\Delta x, \Delta y, \Delta z$ respectively represent the distances along the x, y, z axes from the key point (x', y', z') to point (x, y, z) within the temporal and spatial local region, while Δl denotes the inter-frame distance between the key point (x', y', z') and other points (x, y, z) .

After feature extraction through the point cloud convolutional layer, the feature information of each token in the point cloud video is obtained, which facilitates subsequent spatial and temporal pruning of the point cloud video within the transformer framework.

2) *Transformer Layer*: The transformer layers take the output of the point cloud convolutional layer as input. The main task of the transformer layer is to extract fine-grained feature of point cloud videos, and perform temporal as well as spatial pruning. To enhance the modeling capacity and robustness of the transformer, we incorporate the multi-head attention mechanism into the processing of point cloud videos. The multi-head attention mechanism employs multiple attention heads in each transformer layer to extract feature information from different directions, shown as Figure 3.

In the transformer architecture, we adopt the Gaussian Error Linear Units (GELU) activation function as the activation function for our method in this paper, and it is defined as follows:

$$xP(X \leq x) = x\Phi(x), \quad (4)$$

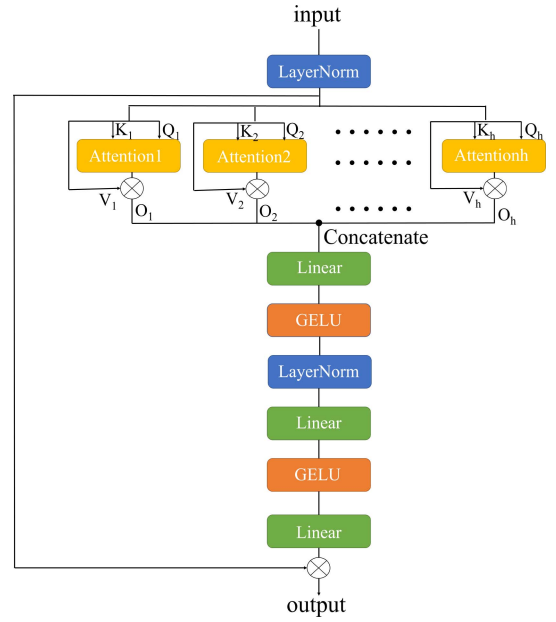


Fig. 3. Structure of transformer layer with multi-head attention.

where $\Phi(x)$ represents the cumulative distribution of the Gaussian normal distribution of x , that is,

$$\Phi(x) = \int_{-\infty}^x \frac{e^{-\frac{(X-\mu)^2}{2\sigma^2}}}{\sqrt{2\pi}\sigma} dX. \quad (5)$$

GELU can help us obtain more attention and importance information, reducing the likelihood of pruning errors during the pruning process.

In transformer layer, first perform attention mechanism to obtain the queries, keys, values, and attention scores. Let $\mathcal{T}$ be the input of the transformer layer, $Q \in \mathbb{R}^{C^q \times LN}$ be queries, $K \in \mathbb{R}^{C^k \times LN}$ be keys and $V \in \mathbb{R}^{C^v \times LN}$ be values. According to [40], the attention score is computed by:

$$Q = W_Q \cdot \mathcal{T}, \quad K = W_K \cdot \mathcal{T}, \quad V = W_V \cdot \mathcal{T}, \quad (6)$$

$$S = \frac{Q^T K}{\sqrt{d_K}}, \quad (7)$$

$$R = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q^T K}{\sqrt{d_K}}\right) \cdot V, \quad (8)$$

where $W_Q \in \mathbb{R}^{C^q \times C}$, $W_K \in \mathbb{R}^{C^k \times C}$, and $W_V \in \mathbb{R}^{C^v \times C}$ are the weights of Q , K , and V , respectively. The operation $\cdot$ represents matrix multiplication. $S \in \mathbb{R}^{LN \times LN}$ is the result of scaling the matrix multiplication of Q and K , representing the attention scores between tokens. The d_K represents the dimension of keys, which is used to scale the attention scores. $R \in \mathbb{R}^{C^v \times LN}$ represents the corresponding weighted attention scores of tokens.

By computing attention scores in the transformer layer, we can further utilize these attention scores in the importance scores for pruning, achieving dual-scale pruning at both temporal and spatial levels.

In summary, in our paper, the transformer layer mainly does the following things. Initially, attention scores are computed

for the fused frames that have incorporated information from previous and subsequent frames, following the processing by the point cloud convolutional layers. Subsequently, an importance score allocation block (ISAB) is employed to calculate the pruning importance scores for each input token and the fusion frame, respectively. Once the importance scores for each token and fusion frame are computed, a judge pruning layer block (JPLB) comes into play to establish whether each transformer layer is subject to temporal-level pruning or spatial-level pruning. After ascertaining the pruning type, temporal pruning block (TPB) and spatial pruning block (SPB) are employed to perform pruning at the respective time and spatial scales. Ultimately, the output comprises the pruned fusion frames and token information. The specific methodological details will be elaborated in this section.

3) *Multilayer Perceptron Layer*: Similar to most of neural networks, the Multi-Layer Perceptron consists of layers such as LayerNorm, linear layers, GELU activation function layers, and hidden layers. MLP achieves tasks such as data classification by applying non-linear transformations and feature extraction to input data. During the training process, MLP optimizes network parameters by using the backpropagation algorithm to make the network output as close as possible to true labels or targeted values. That is, preprocessed data is fed into the MLP, and ultimately, a scalar representing action prediction is obtained to assess the targeted network's training accuracy. We adjust the number of layers and neurons in MLP based on the complexity of tasks and the characteristics of data to obtain better performance.

Specifically, we leverage the transformer for point cloud video pruning. Depending on the number of transformer layers utilized, distinct pruning strategies are executed. Our approach is referred to as the spatio-temporal pruning method.

B. The Spatio-Temporal Pruning Method

With the attention mechanism in transformer, we obtain attention scores between tokens. Using these attention scores, we calculate the importance scores of each token and each frame. To minimize redundancy and achieve efficient computation for point cloud videos, we propose a spatio-temporal pruning method which will be performed in the transformer layers. Firstly, it prunes frames with low importance scores in the temporal dimension. Then, it prunes tokens with low importance scores in the spatial dimension. We do not perform pruning in the first and last transformer layers to keep accuracy.

In order to model the relationships between frames in point cloud videos and address the correctness of importance scores for these frames, we introduce the frame-wise relation weight matrix.

1) *Frame-Wise Relation Weight Matrix*: Using the self-attention mechanism in transformer, we can obtain importance scores of each token with respect to other tokens in the entire video. We use X_l^h to represent the set of tokens obtained after the processing of the l -th frame by the h -th attention head. As each token has relationship with frames, we propose frame-wise relation weight matrix $W_s \in \mathbb{R}^{LN \times LN}$, and $w_{st}^{ij} \in W_s$

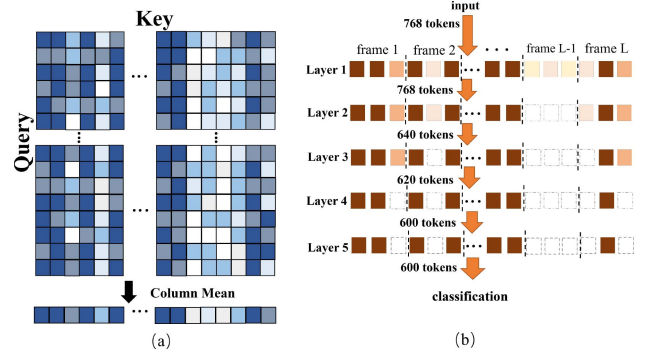


Fig. 4. (a) Example of weighted attention score matrix for an attention head, where a darker color of a cell indicate higher attention. (b) Illustration of token and frame pruning. Dashed lines are used to separate different frames. Unimportant tokens (white cells) and frames are pruned based on temporal and spatial scales as tokens and frames pass through transformer layers.

represents the weight of the attention score between token i in frame s and token j in frame t . Tokens in the same frame have the same weight. The weights are higher for adjacent frames, and conversely, as the distance between frames increases, the weights of the tokens between them become smaller. Let $W'_s \in \mathbb{R}^{LN \times LN}$ be the normalized matrix of W_s , $W_{st}^{ij} \in W'_s$, and it is calculated by:

$$W_{st}^{ij} = \frac{w_{st}^{ij}}{\sum_{s=1}^L \sum_{i=1}^N w_{st}^{ij}}. \quad (9)$$

We use a weighted attention score matrix $A \in \mathbb{R}^{LN \times LN}$ to record the attention scores among all tokens from one head. A is computed by:

$$A = W'_s \odot \text{softmax} \left(\frac{\tau^T W_Q^T W_K T}{\sqrt{d_K}} \right), \quad (10)$$

where $A_{st}^{ij} \in A$ represents the weighted attention score between token i in frame s and token j in frame t . $\odot$ is the Hadamard multiplication.

To prune frames and tokens, we give each token an importance score. The importance score S_t^j for token j in t -th frame as follows:

$$S_t^j = \frac{1}{H} \sum_{h=1}^H g_t^j(h), \quad (11)$$

$$g_t^j(h) = \frac{1}{L} \frac{1}{N} \sum_{s=1}^L \sum_{i=1}^N A_{st}^{ij}(h), \quad (12)$$

where $A_{st}^{ij}(h) \in A(h)$, and $A(h)$ represents the weighted attention score matrix for the h -th head. A token with a higher importance score is considered to be more important and will receive greater attention; a token with a lower importance score is less important and will receive less attention. Figure 4 (a) shows an example of the weighted attention score matrix for an attention head.

Figure 4 (b) illustrates a rough process of token and frame pruning at different transformer layers. We will explain the details of temporal and spatial pruning below.

2) *Pruning on Temporal Scale*: In the temporal scale, we mainly prune frames of 3D point videos. Likewise, we give each frame an important score, which is the summation of important scores of all tokens of the frame. Let S_t be the importance score of frame t , and we have

$$S_t = \frac{1}{N} \sum_{j=1}^N S_t^j, \quad t \in \{1, 2, 3, \dots, L\}, \quad (13)$$

where N is the total number of tokens in a frame. A frame with a higher importance score is considered to be more important and will receive greater attention; a frame with a lower importance score is less important and will receive less attention.

We perform frame pruning based on important scores of frames. We apply the top- K method to determine which frames are kept and which frames are pruned. That is, if sort all frames according to their important scores from high to low, then we keep the first K frames and discard the rest frames. Denote O_t to be the set of the K kept frames, and we have

$$O_t = \text{topK}(S_1, S_2, \dots, S_L). \quad (14)$$

3) *Pruning on Spatial Scale*: In the spatial scale, we mainly prune tokens of 3D point videos. After pruning frames, we perform token pruning for the kept frames based on the important score of tokens. Also, we utilize the top- K method to determine which tokens are remained and which tokens are pruned. The K tokens with the K largest important scores among all tokens are kept, and the other tokens are pruned. Suppose that the rest frames are $i_1, i_2, \dots, i_K$, $1 \leq i_k \leq L$, $1 \leq k \leq K$. Denote O_s to be the set of the K kept frames, and we have

$$O_s = \text{topK}(s_{i_1}^1, \dots, s_{i_1}^N, \dots, s_{i_K}^1, \dots, s_{i_K}^N), \quad (15)$$

where $s_{i_k}^j$ represents the importance score for token j of frame i_k .

Algorithm 1 shows our designed spatio-temporal pruning method. It describes how to prune frames and tokens in transformer layers for point cloud videos, which is our main work.

IV. EXPERIMENT

In this section, we will show the effectiveness and efficiency of our proposed method STP by experiments.

A. Experimental Details

1) *Dataset Introduction*: The MSR Action 3D dataset [41] comprises sequences of human actions, encompassing a total of 20 action categories performed by 10 participants. Each action is executed either 2 or 3 times per participant. The dataset encompasses 567 sequences of depth images, totaling 23,000 frames, captured at a resolution of 640x240 using the Kinect v1 depth sensor. For this study, the initial ten participants' executions of actions are utilized as the training set for network training, while the subsequent ten participants' executions are reserved as the test set for network evaluation.

Algorithm 1 The Spatio-Temporal Pruning (STP)

Require: tokens from point cloud convolution, the value K for the top- K method, the number of transformer layers C .
Ensure: the rest tokens after pruning.

- 1: **for** transformer layer $c = 1$ to C **do**
- 2: **if** c is a temporal pruning layer **then**
- 3: calculate W'_s by Eq. (9);
- 4: calculate A by Eq. (10);
- 5: calculate important scores of tokens by Eq. (11);
- 6: calculate important scores of frames by Eq. (13);
- 7: perform frame pruning according to Eq. (14);
- 8: **else if** c is a spatial pruning layer **then**
- 9: calculate W'_s by Eq. (9);
- 10: calculate A by Eq. (10);
- 11: calculate important scores of all frames by Eq. (11);
- 12: perform token pruning according to Eq. (15);
- 13: **else**
- 14: continue;
- 15: **end if**
- 16: **end for**
- 17: **return** the rest tokens after pruning.

TABLE II
HARDWARE AND SOFTWARE CONFIGURATION FOR EXPERIMENT

Hardware and software configuration	
Operating system	CentOS Linux release 7.6.1810
CPU	Intel(R) Xeon(R) Gold 5218R CPU @ 2.10GHz
GPU	Intel(R) Nvidia GeForce RTX 2080 Ti(12GB)*4
Memory	125GB
PyTorch version	3.7.12
Torch version	10.2
CUDA version	11.3

The NTU RGB+D 60 dataset [42] used in this study is an action recognition dataset consisting of 60 action categories and a total of 56,880 samples. The dataset includes 40 categories of daily actions, 9 categories of health-related actions, and 11 categories of interactions between two people. The dataset defines two types of evaluation: cross-subject and cross-view. The cross-subject evaluation splits the 40 performers into training and test groups. Each group consists of 20 performers. The overall dataset is divided into training and testing sets based on the standard of different individuals performing actions. Specifically, 20 individuals are allocated to the training set, resulting in a total of 40,320 samples. Another set of 20 individuals are assigned to the testing set, comprising a total of 16,560 samples. The training set consisted of individuals with the following identification numbers: 1, 2, 4, 5, 8, 9, 13, 14, 15, 16, 17, 18, 19, 25, 27, 28, 31, 34, 35, and 38. On the other hand, the testing set included individuals with the following identification numbers: 3, 6, 7, 10, 11, 12, 20, 21, 22, 23, 24, 26, 29, 30, 32, 33, 36, 37, 39, and 40. The cross-view evaluation uses all the samples from camera 1 for testing and samples from cameras 2 and 3 for training.

2) *Parameter Settings*: In terms of hardware support for the experiments, PyTorch is used as the training platform. The

TABLE III
COMPARISON OF ACCURACY (%) BETWEEN FOUR METHODS ON MSR ACTION 3D

Method	Temporal scale	Spatial scale	Clip level Accuracy		Video level Accuracy
			Top-1	Top-5	Top-1
PST-transformer [13]			84.705	95.351	93.37
TOP	✓		84.429	95.332	92.64
SOP		✓	84.746	95.031	92.55
STP (ours)	✓	✓	85.124	96.367	93.57

TABLE IV
COMPARISON OF ACCURACY (%) BETWEEN FOUR METHODS ON NTU RGB+D 60

Method	Temporal scale	Spatial scale	Clip level Accuracy		Video level Accuracy	
			Top-1	Top-5	Subject	View
PST-transformer [13]			84.765	96.722	91.0	96.1
TOP	✓		84.235	96.566	90.9	95.4
SOP		✓	83.925	95.674	89.7	95.1
STP (ours)	✓	✓	84.476	96.774	91.5	96.4

model and dataset are loaded onto the server for training. The connection between the local compiler and the remote server is established to build the corresponding model. The specific environment is shown in Table II.

Regarding the parameter settings of the experiments, for the MSR Action 3D dataset, the training iterations, batch size, initial learning rate, data loaders, initial iterations for learning rate warm-up, and weight decay are set to be 70, 14, 0.01, 10, 10, 0.0001, respectively. For the NTU RGB+D 60 dataset, the training iterations, batch size, initial learning rate, data loaders, initial iterations for learning rate warm-up, and weight decay are set to be 15, 24, 0.01, 16, 10, and 0.0001, respectively. The optimizer for training is SGD. The adopted loss function is the cross-entropy, and its specific formula is shown as follows:

$$CE(p, q) = - \sum_{i=1}^D p_i \log(q_i), \quad (16)$$

where D is the number of classes for classification, p and q are sets of p_i and q_i , respectively. p_i represents the true value of class i , and q_i represents the predicted value of class i .

3) *Evaluation Metrics*: As plenty of relative work, we use three common metrics to evaluate our proposed method: the average inference time, Top-5 accuracy, and Top-1 accuracy.

Top-1 accuracy refers to the highest accuracy among all predictions made by a model concerning the true label. Essentially, it indicates whether the most probable class predicted by the model matches the actual label. In other words, top-1 accuracy represents the ratio of samples with correct predictions at the top-1 position to the total number of samples used for prediction. Top-5 accuracy allows the model to match the true label within the top five highest probability predictions. The average inference time is obtained

TABLE V
COMPARISON OF AVERAGE INFERENCE TIME (S) BETWEEN FOUR METHODS ON MSR ACTION 3D AND NTU RGB+D 60

Method	Average inference time	
	MSR Action 3D	NTU RGB+D 60
PST-Transformer [13]	0.1932	0.1265
TOP	0.1376	0.0998
SOP	0.1401	0.1032
STP (ours)	0.1344	0.0908

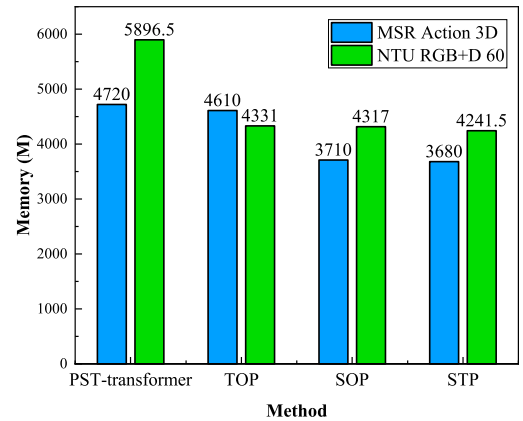


Fig. 5. The GPU memory usage for four methods on different datasets.

by calculating the time taken by the network for a single batch throughout of the entire inference process. The formulas for calculating top-1 accuracy, top-5 accuracy, and the average inference time are as follows:

$$Acc_1 = \frac{r_1}{T} \times 100\%, \quad (17)$$

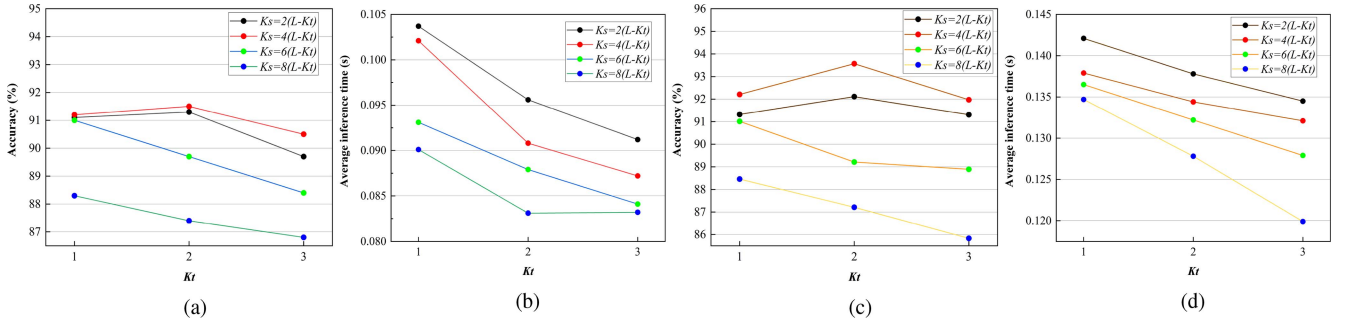


Fig. 6. The impact of different values of K_t and K_s on the accuracy and the average inference time. (a) and (b) are for the MSR Action 3D dataset; (c) and (d) are used for the NTU RGB+D 60 dataset.

$$Acc_5 = \frac{r_5}{T} \times 100\%, \quad (18)$$

$$\overline{Inf} = \frac{t_s - t_e}{N_b}, \quad (19)$$

where r_1 and r_5 represent the number of samples correctly predicted in top-1 and top-5, respectively. T denotes the total number of samples, t_s and t_e denote the start and end time of the inference stage, and N_b is the number of batches.

B. Analysis of Experimental Results

To verify the effectiveness and correctness of the proposed method, comparative experiments and ablation experiments are conducted for different methods and parameter settings.

1) *Comparison Experiment*: As there are currently few approaches based on transformer for pruning point cloud video and they are not suitable to solve our problem, we choose three methods to compare with our method. These methods are:

- PST-transformer [13]: it is designed for classifying actions in point cloud videos with considering spatial and temporal information but no pruning. It is regarded as a baseline.
- Temporal-scale pruning only method (TOP): Only point frames are pruned.
- Spatial-scale pruning only method (SOP): Only point cloud tokens are pruned.

We first compare our proposed method with other three methods in aspects of accuracy as well as average inference time, and then the occupied memory space. We maintain consistent parameter settings across different approaches. To ensure a comparable pruning ratio, we regard PST-transformer as the baseline, set $K_t = 3$ and $K_s = 0$ for TOP, as well as let $K_t = 0$ and $K_s = 14(L - K_t)$ for SOP.

Table III shows the classification accuracy obtained by four approaches on the dataset of MSR Action 3D. It is evident that our proposed STP outperforms the other three approaches both in terms of video-level and clip-level accuracy. This is due to the more precise pruning achieved by our approach, ensuring the retention of crucial point cloud information while eliminating redundancy and noise in the point cloud videos. Compared to PST-transformer, our method exhibits improvements of 0.369% in clip-level top1 accuracy, 1.008% in clip-level top5 accuracy, and 0.2% in video-level top1

TABLE VI
THE IMPACT OF FRAME-WISE RELATION WEIGHT MATRIX ON EXPERIMENTAL ACCURACY (%) AND AVERAGE INFERENCE TIME (S)

Dataset	Method	Accuracy	Average inference time
MSR Action 3D	STP(w/o)	93.36	0.1373
	STP(w)	93.57	0.1344
NTU RGB+D 60	STP(w/o)	91.1	0.0954
	STP(w)	91.5	0.0908

w/: with the frame-wise relation weight matrix.

w/o: without the frame-wise relation weight matrix.

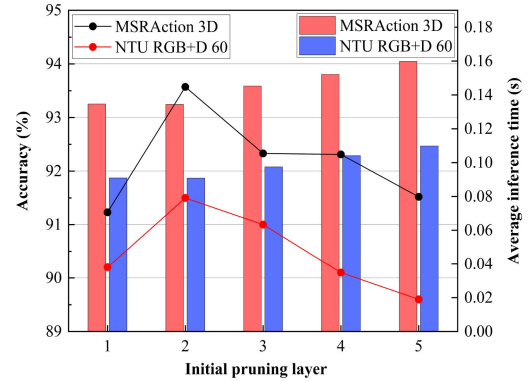


Fig. 7. The impact of pruning initial layers on accuracy and average inference time.

accuracy. PST-transformer, which does not undergo pruning, exhibits lower accuracy.

Table IV shows the similar results to Table III in classification accuracy obtained by four approaches on the dataset of NTU RGB+D 60. Although it is observed that PST-transformer finishes the best top-1 accuracy among all methods, on the whole, our proposed method outperforms the other three methods.

Table V shows the average inference time obtained by four approaches across two given datasets. It is obvious that the proposed STP outperforms other approaches. Compared to PST-transformer, STP exhibits a reduction of 28% and 30% in NTU RGB+D 60 and MSR Action 3D, respectively.

Figure 5 shows the comparison between our approach and others in terms of GPU memory consumption. It is apparent

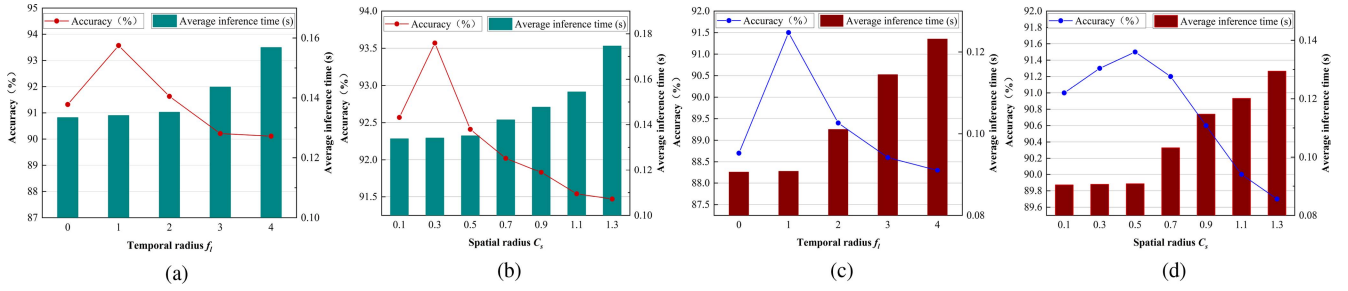


Fig. 8. Influence of the temporal radius f_l and spatial radius C_s . The MSR Action 3D dataset is used on (a) and (b), the NTU RGB+D 60 dataset is used on (c) and (d).

that the proposed STP method significantly mitigates the pressure of GPU memory usage, and reduce it by nearly 1100M on MSR Action 3D dataset and 1600M on NTU RGB+D 60 dataset. As a result, our method can be deployed on devices with smaller GPU memory, facilitating the accomplishment of action classification tasks.

2) *Ablation Experiments:* Also, we do ablation experiments to demonstrate the correctness of the parameter settings for our proposed method. The following parameter settings are considered:

- K-value setting
- Frame-wise relation matrix setting
- The number of initial transformer layer for pruning setting
- Temporal radius f_l and spatial radius C_s settings

a) *K-value setting:* We first select different temporal values of K_t and spatial pruning values of K_s , and then compared the effects of different combinations of K_t and K_s values on the pruning results. Figure 6 shows the changes in the video classification accuracy and average inference time under different K-values. Taking all factors into account, we choose to perform pruning at $K_t = 2$ and $K_s = 4(L - K_t)$, where L represents the number of input frames in experiments and is set to be 12. As it results in the highest precision and reduces the inference time significantly. When a few of tokens are pruned, the redundancy of tokens leads to lower accuracy. When a great many tokens are pruned, although the inference time decreases, it affects tokens with high importance. Pruning off these important tokens leads to a decline in precision, which is unfavorable for classifying tasks.

b) *Frame-wise relation matrix setting:* The design of the frame-wise relationship matrix is primarily aimed at obtaining the correlation between frames and modeling temporal information during the importance score calculation of fusion frames. Table VI shows the comparison of video classification accuracy and average inference time with and without considering the frame-wise relationship matrix while keeping other conditions unchanged. It can be seen that considering the frame-wise relationship matrix brings higher accuracy and lower average inference time.

c) *The number of initial transformer layer for pruning setting:* To validate the correctness of the initial pruning transformer layer setting, we conduct experiments by pruning at different transformer layers and assessing the performance of our proposed method. Figure 7 shows the accuracy and

average inference time obtained by initial pruning at different transformer layer. It can be observed that when initial pruning starts with the second transformer layer, the accuracy is the largest among five transformer layers for all four methods. Because pruning begin with the first layer directly often results in the loss of important token features that have not yet been learned by the network. This leads to a loss of feature information during subsequent pruning of the transformer layers and a further decrease in accuracy. Also, the average inference time obtained by starting initial pruning at transformer layers 1 and 2 are quite close, and they are smaller than that obtained by starting initial pruning at transformer layers 3, 4 and 5. Therefore, we choose initial pruning at the transformer layer 2 in our experiments.

d) *Temporal radius f_l and spatial radius C_s settings:* In the point cloud convolutional layer, the values of the temporal radius f_l and spatial radius C_s influence the modeling of spatio-temporal local structural features. f_l controls the temporal dynamics modeling of point cloud videos. Figure 8(a) and Figure 8(c) shows the changes of the classification accuracy and the average inference time when f_l varies from 0 to 4 for the MSR Action 3D dataset and the NTU RGB+D 60 dataset, respectively. It is obvious that the accuracy is the largest when $f_l = 1$ for two datasets. In comparison with $f_l = 0$ which means that only spatial structures are captured, $f_l = 1$ indicates that the temporal local structures of three frames can be captured, enabling modeling of spatio-temporal correlations, containing more information, facilitating subsequent self-attention by transformers, and consequently achieving a higher accuracy. However, as f_l exceeds 1, the accuracy gradually decreases. This is attributed to points entering and leaving the video, especially in long videos. Introducing longer temporal radius unavoidably introduces noise. Also, it is observed that with f_l increasing, the average inference time gradually increases due to the need to model more frames, impacting the inference time. The average inference time for $f_l = 1$ is very close to that for $f_l = 0$. All things considered, we set $f_l = 1$ in our experiments for two datasets.

The spatial radius C_s controls the region for modeling spatial structures. Figure 8(b) and Figure 8(d) shows the changes of the accuracy and the average inference time when C_s varies from 0.1 to 1.3 with an increment of 0.2 for two different datasets, respectively. We obtain the similar results for C_s to that for f_l . Therefore, we set $C_s = 0.3$ for the MSR Action 3D dataset and $C_s = 0.5$ for the NTU RGB+D 60 dataset in

our experiments. With an increase in Cs, the spatial area for feature extraction expands, resulting in a gradual increase in average inference time.

V. CONCLUSION

With the development of society and the consumer market focused on user experience, the amount of 3D point cloud data is increasing rapidly, and the demand for real-time devices is also increasing. In this paper, we design a spatio-temporal pruning method STP applied to transformer-based PCNN for 3D point cloud video processing in consumer-centric applications. By considering spatial and temporal scales, fine-grained pruning of point cloud videos can be achieved, enabling faster data processing for practical applications such as AR, VR, and autonomous driving. That is, temporal and spatial pruning allow for a more refined trimming process, eliminating redundancy and noise in point cloud videos, thereby enhancing the accuracy and speed of the network in action recognition. In addition, we adopt a parameter-free pruning method to prevent complexity and introduce a frame-interrelation weight matrix for time modeling to avoid loss of temporal information. Our method can be easily deployed on transformer-based PCNN to meet consumer demands for real-time requirements and broader applications in mobile edge computing. Experimental results demonstrate the effectiveness and efficiency of our proposed method.

The application of point cloud pruning in the mobile edge computing domain faces numerous challenges and constraints. For instance, the high computational complexity of point clouds, stemming from their enormous data volume, poses substantial difficulties in practical pruning processes. Moreover, the point cloud data collected by devices might contain significant noise and errors, further impacting the accuracy of pruning. Additionally, the high storage and processing demands of point cloud data necessitate pruning methods that reduce data volume and storage needs while maintaining data quality. These issues call for more suitable point cloud pruning approaches tailored for different scenarios, and they are our future research.

REFERENCES

- [1] Y. Wang et al., "3-DV: 3-D dynamic voxel for action recognition in depth video," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 511–520.
- [2] J. Liu and D. Xu, "GeometryMotion-Net: A strong two-stream baseline for 3-D action recognition," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 12, pp. 4711–4721, Dec. 2021.
- [3] P. Cambou, *3-D Imaging and Sensing Technologies: An Expected Explosion within the Consumer Market Segment*, Yole Group, Lyon, France, 2017.
- [4] C. Chen, K. Li, S. G. Teo, X. Zou, K. Li, and Z. Zeng, "Citywide traffic flow prediction based on multiple gated spatio-temporal convolutional neural networks," *ACM Trans. Knowl. Disc. Data*, vol. 14, no. 4, pp. 1–23, 2020.
- [5] C. Chen, K. Li, W. Wei, J. T. Zhou, and Z. Zeng, "Hierarchical graph neural networks for few-shot learning," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 1, pp. 240–252, Jan. 2021.
- [6] Y. Cui et al., "Deep learning for image and point cloud fusion in autonomous driving: A review," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 722–739, Feb. 2022.
- [7] M. Dhouibi, A. K. Ben Salem, A. Saidi, and S. Ben Saoud, "Accelerating deep neural networks implementation: A survey," *IET Comput. Digit. Techn.*, vol. 15, no. 2, pp. 79–96, 2021.
- [8] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "AMC: AutoML for model compression and acceleration on mobile devices," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 784–800.
- [9] R. B. Rusu and S. Cousins, "3-D is here: Point cloud library (PCL)," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2011, pp. 1–4.
- [10] J. Guo, J. Liu, and D. Xu, "JointPruning: Pruning networks along multiple dimensions for efficient point cloud processing," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 6, pp. 3659–3672, Jun. 2022.
- [11] J. Guo, J. Liu, and D. Xu, "3-D-Pruning: A model compression framework for efficient 3-D action recognition," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 12, pp. 8717–8729, Dec. 2022.
- [12] H. Fan, Y. Yang, and M. Kankanalli, "Point 4D transformer networks for spatio-temporal modeling in point cloud videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 14204–14213.
- [13] H. Fan, Y. Yang, and M. Kankanalli, "Point spatio-temporal transformer networks for point cloud video modeling," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 2, pp. 2181–2192, Feb. 2022.
- [14] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [15] M. Zhu, Y. Tang, and K. Han, "Vision transformer pruning," 2021, *arXiv:2104.08500*.
- [16] F. Yu, K. Huang, M. Wang, Y. Cheng, W. Chu, and L. Cui, "Width & depth pruning for vision transformers," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 3143–3151.
- [17] H. Yu and J. Wu, "A unified pruning framework for vision transformers," *Sci. China Inf. Sci.*, vol. 66, no. 7, pp. 1–2, 2023.
- [18] J. Zhang et al., "MiniViT: Compressing vision transformers with weight multiplexing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12145–12154.
- [19] C. Zheng et al., "SAViT: Structure-aware vision transformer pruning via collaborative optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 9010–9023.
- [20] T. Chen, Y. Cheng, Z. Gan, L. Yuan, L. Zhang, and Z. Wang, "Chasing sparsity in vision transformers: An end-to-end exploration," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 19974–19988.
- [21] Y. Tang et al., "Patch slimming for efficient vision transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12165–12174.
- [22] Y. Xu et al., "Evo-ViT: Slow-fast token evolution for dynamic vision transformer," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 2964–2972.
- [23] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, and C.-J. Hsieh, "DynamicViT: Efficient vision transformers with dynamic token sparsification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 13937–13949.
- [24] Y. Liang, C. Ge, Z. Tong, Y. Song, J. Wang, and P. Xie, "Not all patches are what you need: Expediting vision transformers via token reorganizations," 2022, *arXiv:2202.07800*.
- [25] X. Zhou et al., "Digital twin enhanced federated reinforcement learning with lightweight knowledge distillation in mobile networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3191–3211, Oct. 2023.
- [26] X. Zhou et al., "Information theoretic learning-enhanced dual-generative adversarial networks with causal representation for robust OOD generalization," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Nov. 17, 2023, doi: [10.1109/TNNLS.2023.3330864](https://doi.org/10.1109/TNNLS.2023.3330864).
- [27] X. Zhou et al., "Hierarchical federated learning with social context clustering-based participant selection for Internet of Medical Things applications," *IEEE Trans. Comput. Soc. Syst.*, vol. 10, no. 4, pp. 1742–1751, Aug. 2023.
- [28] X. Zhou, W. Liang, K. I.-K. Wang, and L. T. Yang, "Deep correlation mining based on hierarchical hybrid networks for heterogeneous big data recommendations," *IEEE Trans. Comput. Soc. Syst.*, vol. 8, no. 1, pp. 171–178, Feb. 2021.
- [29] X. Zhou et al., "Decentralized p2p federated learning for privacy-preserving and resilient mobile robotic systems," *IEEE Wireless Commun.*, vol. 30, no. 2, pp. 82–89, Apr. 2023.
- [30] H. Yang, H. Yin, M. Shen, P. Molchanov, H. Li, and J. Kautz, "Global vision transformer pruning with hessian-aware saliency," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 18547–18557.
- [31] Z. Kong et al., "Spvit: Enabling faster vision transformers via soft token pruning," 2021, *arXiv:2112.13890*.
- [32] Z. Song, Y. Xu, Z. He, L. Jiang, N. Jing, and X. Liang, "CP-ViT: Cascade vision transformer pruning via progressive sparsity prediction," 2022, *arXiv:2203.04570*.

- [33] M. Fayyaz et al., "Adaptive token sampling for efficient vision transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 396–414.
- [34] H. Yin, A. Vahdat, J. M. Alvarez, A. Mallya, J. Kautz, and P. Molchanov, "A-ViT: Adaptive tokens for efficient vision transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10809–10818.
- [35] H. Ma et al., "PPT: Token-pruned pose transformer for monocular and multi-view human pose estimation," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 424–442.
- [36] D. Graziosi, O. Nakagami, S. Kuma, A. Zaghetto, T. Suzuki, and A. Tabatabai, "An overview of ongoing point cloud compression standardization activities: Video-based (V-PCC) and geometry-based (G-PCC)," *APSIPA Trans. Signal Inf. Process.*, vol. 9, p. e13, Apr. 2020.
- [37] Y. Lee, D. Lee, J. Hong, J. W. Lee, and H. Yoon, "Not all neighbors matter: Point distribution-aware pruning for 3-D point cloud," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 1240–1249.
- [38] Y. Huang et al., "CP3: Channel pruning plug-in for point-based networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 5302–5312.
- [39] M. Li, C. Chen, X. Yang, J. T. Zhou, T. Zhang, and Y. Li, "Toward communication-efficient digital twin via AI-powered transmission and reconstruction," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3624–3635, Nov. 2023.
- [40] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.
- [41] W. Li, Z. Zhang, and Z. Liu, "Action recognition based on a bag of 3-D points," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. Workshops*, 2010, pp. 9–14.
- [42] A. Shahroudy, J. Liu, T.-T. Ng, and G. Wang, "NTU RGB+ D: A large scale dataset for 3-D human activity analysis," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 1010–1019.



Jing Liu received the Ph.D. degree in computer science from Hunan University, Changsha, China, in 2018. She currently works as an Associate Professor with the School of Computer Science and Technology, Wuhan University of Science and Technology. Her research interest includes edge computing, parallel computing, task scheduling, and high-performance computing.



Zhongyao Cheng received the master's degree from the National University of Singapore, Singapore, in 2018. He is a Senior Research Engineer with I2R, A*Star, Singapore. He has provided technical support for academic articles and industrial projects. He has published several papers on deep learning and computer vision. His research interests focus on computer vision and deep learning.



Cen Chen (Senior Member, IEEE) received the Ph.D. degree in computer science from Hunan University, Changsha, China, in 2019. He currently works as a Professor with the School of Future Technology, South China University of Technology. He previously worked as a Scientist with the Institute of Infocomm Research, Agency for Science, Technology and Research, Singapore. He has published more than 60 articles in international conferences and journals on machine learning algorithms and parallel computing, such as MICRO, HPCA, DAC, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, AAAI, ICDM, ICPP, and ICDCS. He is also an Associate Editor of the IEEE TRANSACTIONS ON COMPUTERS.

as MICRO, HPCA, DAC, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, AAAI, ICDM, ICPP, and ICDCS. He is also an Associate Editor of the IEEE TRANSACTIONS ON COMPUTERS.



Ziqian Zeng received the Ph.D. degree in computer science and engineering from The Hong Kong University of Technology in 2021. She is currently an Associate Professor with the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology. Her research interests include efficient inference, fairness, and privacy.



Songyu Xie (Graduate Student Member, IEEE) received the bachelor's degree in information and computing science from the Wuhan University of Science and Technology, Hubei, China, in 2021, where he is currently pursuing the master's degree with the School of Computer Science and Technology. His research interests include Computer vision and distributed computing, machine learning, and deep learning.



Huiping Zhuang (Member, IEEE) received the B.S. and M.E. degrees from the South China University of Technology, Guangzhou, China, in 2014 and 2017, respectively, and the Ph.D. degree from the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore, in 2021. He is currently an Associate Professor with the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology. He has published more than 30 papers, including those in ICML, NeurIPS, CVPR, IEEE

TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS, and IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING. His research interests include deep learning, AI computer architecture, and intelligent robot. He has served as a Guest Editor for *Journal of Franklin Institute*.